

SERVIDOR DE PIPELINES CONCURRENTE

CI-0117 PROGRAMACIÓN PARALELA Y CONCURRENTE · UCR · II-2026

Miguel Fabricio Arias
Santiago Asunción
Andrés Oreamuno

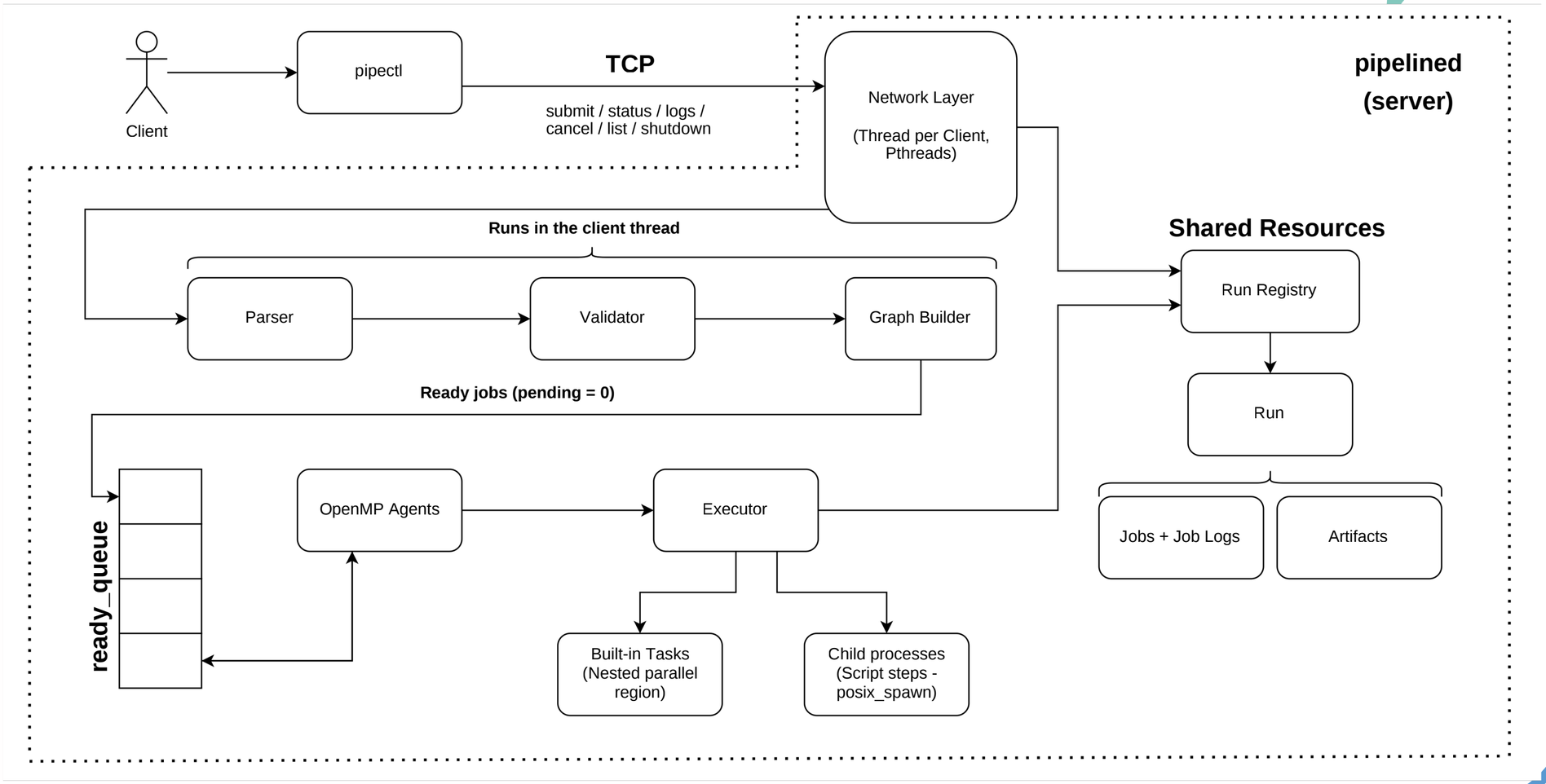
EL PROBLEMA



- Un pipeline: etapas → trabajos → pasos, con dependencias
- Lo que no depende puede correr en paralelo
- Pero los agentes son limitados

- Los trabajos comparten estado, variables, artefactos y bitácoras
- Mal sincronizado, el mismo pipeline da resultados distintos

ARQUITECTURA



ARQUITECTURA

- Capa de red con Pthreads: un hilo por conexión
- Validación: YAML → validador → grafo de dependencias
- Ejecución con OpenMP: cola → agentes → tareas
- Agentes e hilos configurables sin recompilar



CONSTRUCCIÓN DEL GRAFO

- Subconjunto del esquema de Azure Pipelines, leído con libyaml
- Por omisión, cada trabajo depende de toda la etapa anterior
- dependsOn reemplaza en la etapa y agrega en el trabajo
- Cada nodo guarda predecesores, sucesores y un contador pending

VALIDACIÓN



SE REPORTAN TODOS LOS ERRORES A LA VEZ, CON SU LÍNEA



FORMATO INVÁLIDO Y CLAVES DESCONOCIDAS



NOMBRES REPETIDOS Y DEPENDENCIAS QUE NO EXISTEN

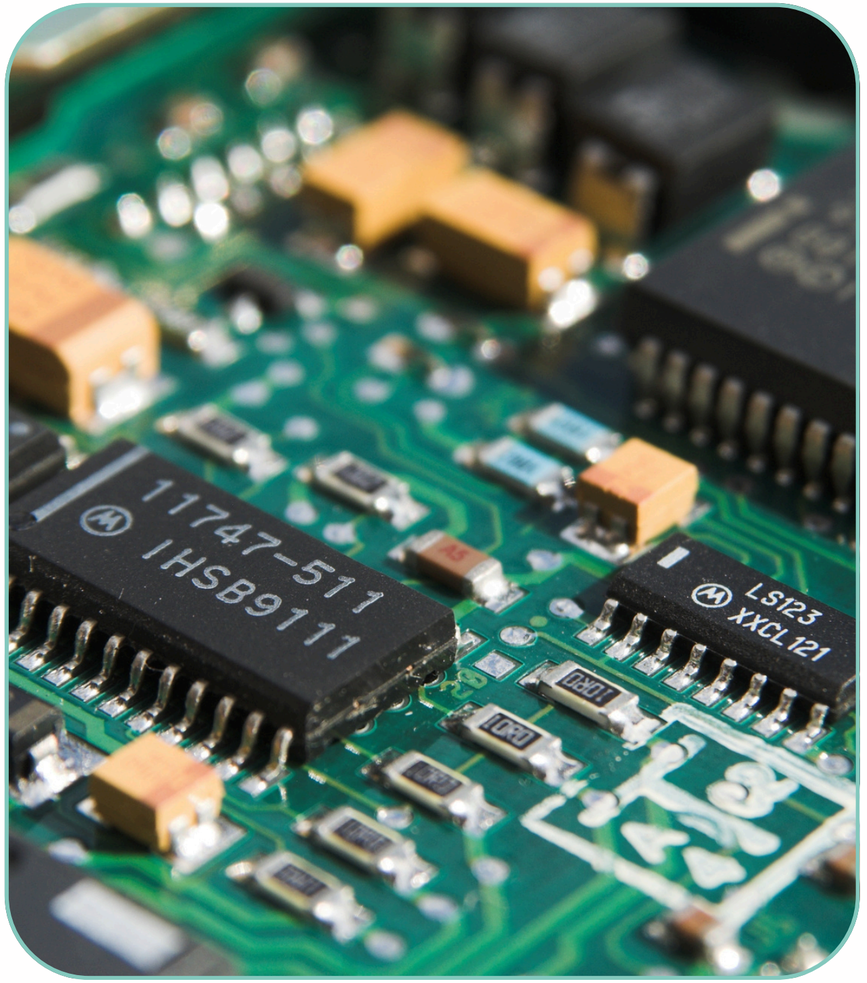


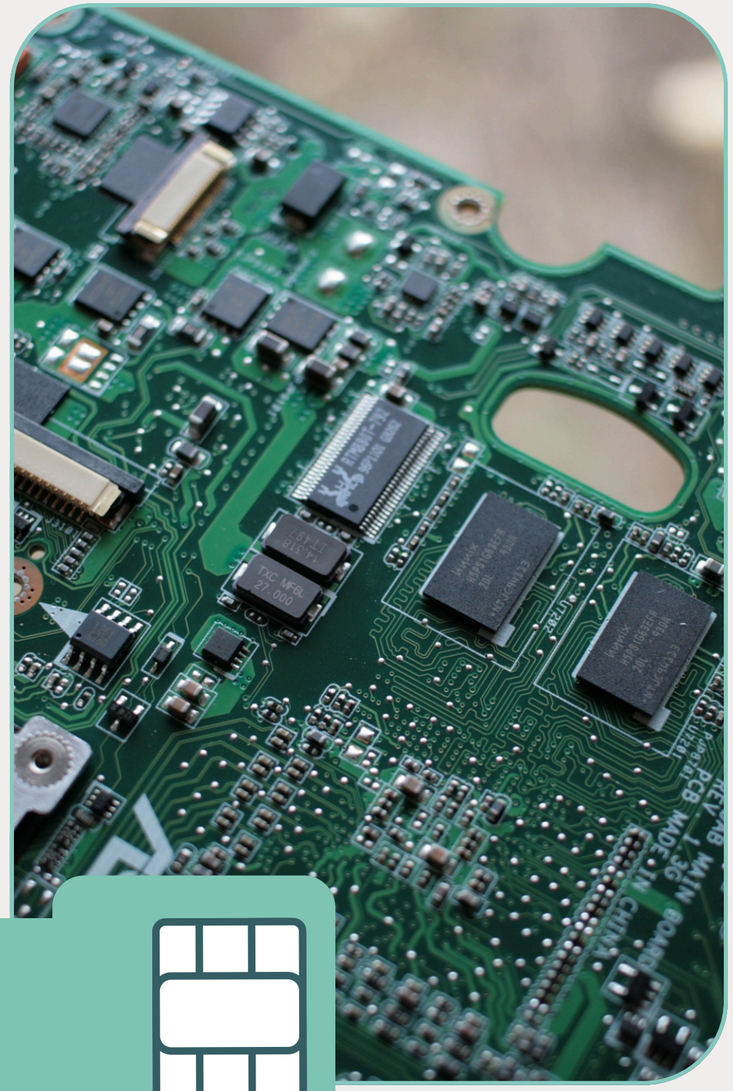
CICLOS CON EL ALGORITMO DE KAHN: DEPENDENCIA CÍCLICA: A -> B -> A

AGENTES Y LÍMITE

- **#pragma omp parallel num_threads(agentes)**

- Una sola región paralela: un hilo es un agente
- Cada agente saca un trabajo de la cola y lo ejecuta
- Nunca hay más trabajos corriendo que agentes
- La cola FIFO es común a todos los pipelines





AVANCE DEL GRAFO

- **#pragma omp atomic capture**

- Al terminar, el trabajo baja el pending de cada sucesor
- El que llega a cero se evalúa y se encola
- succeeded(), failed() y always() deciden si corre o queda skipped
- Un trabajo saltado también libera a sus sucesores

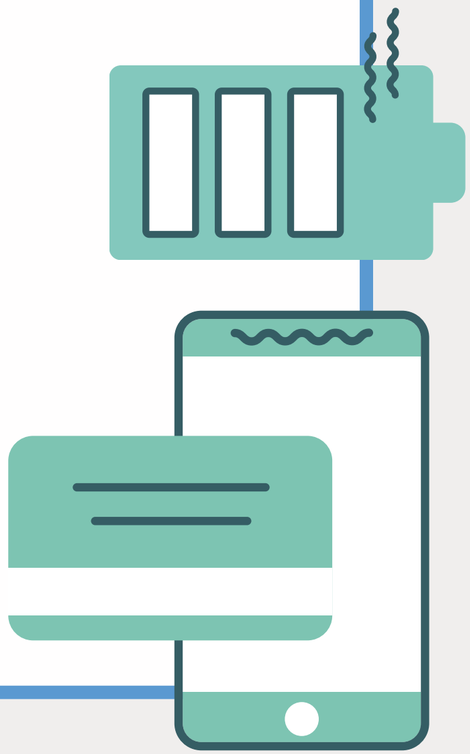


RECURSOS COMPARTIDOS

Recurso	Mecanismo
Cola de trabajos listos	*Mutex y variable de condición*
Contadores pending y remaining	*omp atomic capture*
Estados, tiempos y bitácoras	*omp_lock_t*
Variables entre trabajos	*Sin bloqueo, por orden de escritura*
Artefactos	*Atomicidad de mkdir*

DETERMINISMO

- Si dos predecesores definen la misma variable, gana el último del grafo
- Histogram@1 combina con reduction(+ : histogram[:bins])
- WordCount@1 cuenta por bloques y mezcla en orden de bloque
- Al ordenar, desempate alfabético
- Resultado idéntico con 1, 4 u 8 hilos

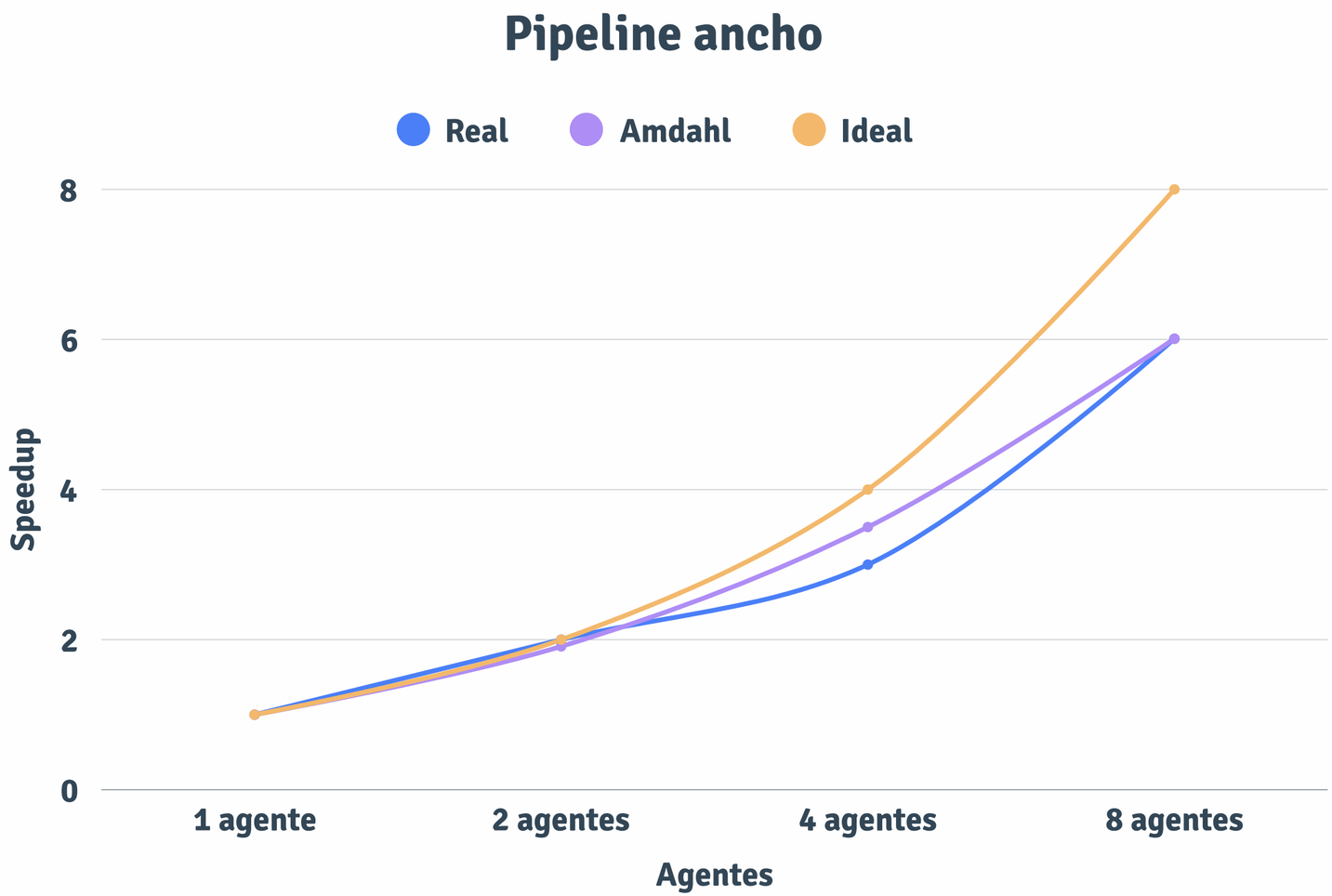


PRUEBAS

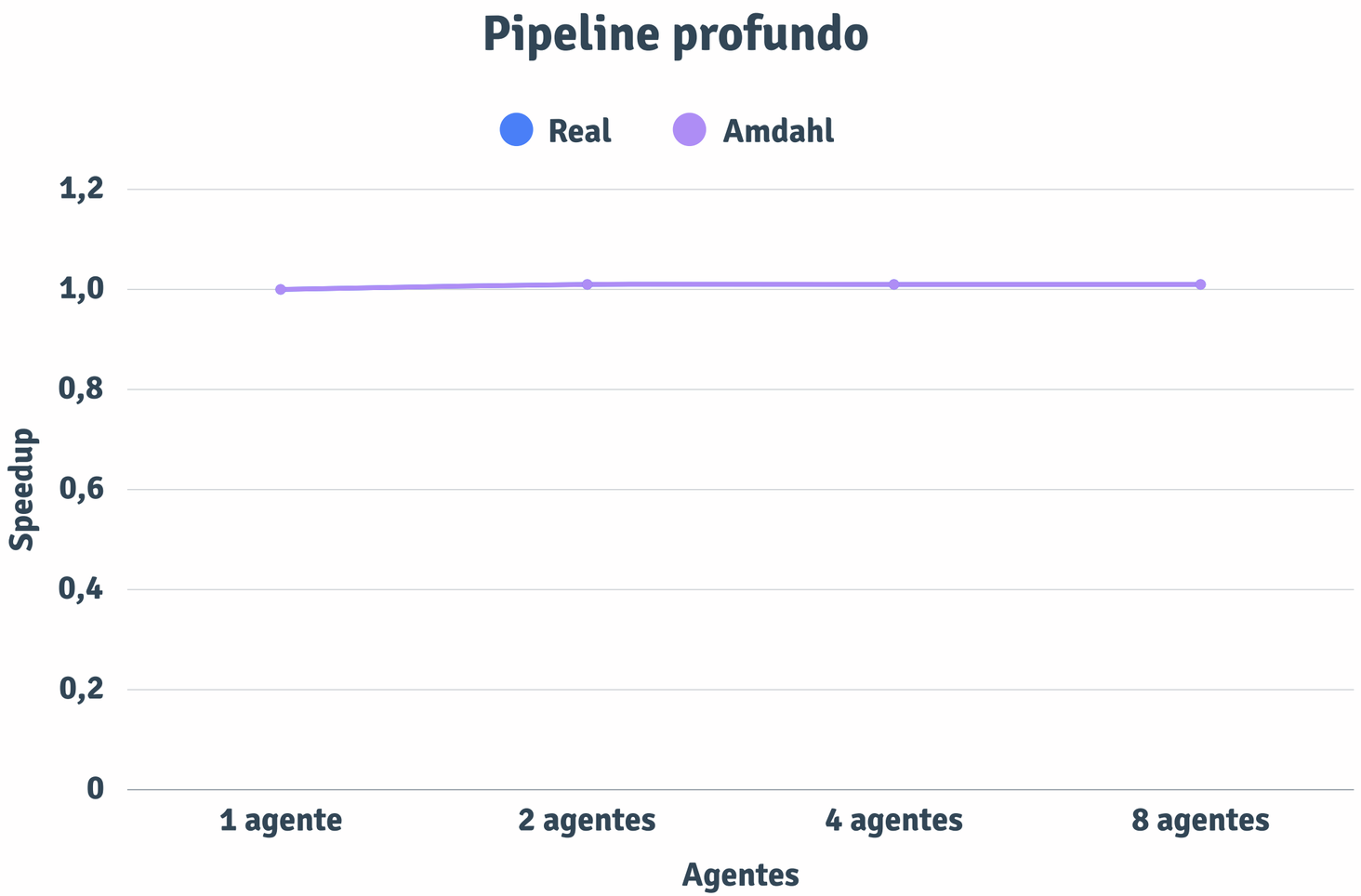
- **make test**

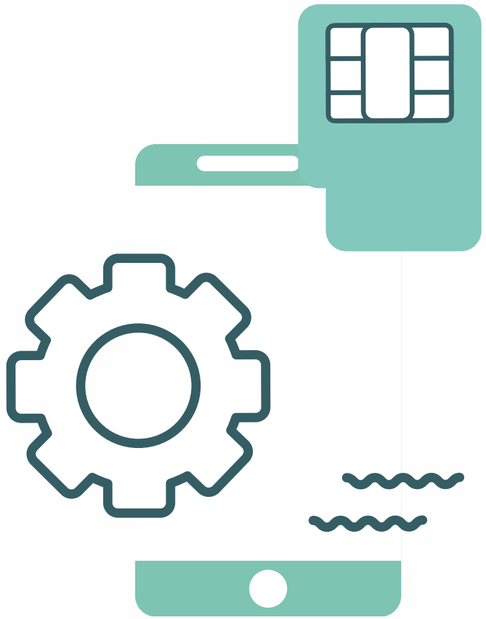
- Unitarias: analizador, tareas, cola y planificador
- Integración: pipeline completo con artefactos y condiciones
- Concurrencia: 10 000 operaciones en la cola, 50 trabajos hacia 1
- Determinismo: mismo resultado con 1 y 4 hilos
- Todo compilado con AddressSanitizer

MÉTRICAS


Pipeline ancho
Real
Amdahl
Ideal
8
6
4
2
0
Speedup
1 agente
2 agentes
4 agentes
8 agentes
Agentes

MÉTRICAS


Pipeline profundo
Real
Amdahl
1,2
1,0
0,8
0,6
0,4
0,2
0
Speedup
1 agente
2 agentes
4 agentes
8 agentes
Agentes



DEMO EN VIVO



CONCLUSIONES

- **Grafo no varía**

El grafo no cambia durante la ejecución: solo los contadores, y esos se resuelven con atómicos

- **Procesos hijos**

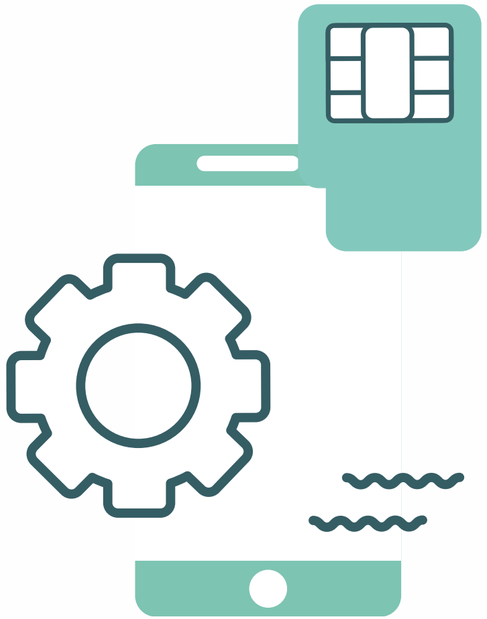
posix_spawn evita el riesgo de crear procesos desde un programa con hilos

- **Determinismo**

Un orden fijo de combinación hace determinista el resultado

- **Limitación**

Queda pendiente enviar el YAML por la red



MUCHAS GRACIAS

